INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	T2.4Fase1APE5. Creación, Lectura y Escritura de Ficheros
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Paredes Acosta Dereck Shair ……..
Asignatura:	Algoritmos y Lógica de programación
Docente:	Ing. José Ruben Caizabuano, M.Sc.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:
Iniciar el manejo de ficheros de texto mediante la implementación de tres ejercicios prácticos de creación, escritura y eliminación de archivos en Java, verificando su funcionamiento con pruebas de escritorio.

Específicos:
- Crear archivos de texto y comprobar su existencia, empleando la clase File y el método createNewFile() de Java.
- Almacenar información ingresada por teclado en un archivo existente sin perder su contenido, empleando las clases Scanner y FileWriter en modo agregar.
- Eliminar archivos, controlar los posibles errores y validar el funcionamiento de cada programa mediante pruebas de escritorio y su ejecución en Java.

2.2 Modalidad
Presencial

2.3 Tiempo de duración
Presenciales: 2
No presenciales: 0

2.4 Instrucciones
1. Leer los enunciados de los tres ejercicios del documento APE 5 e identificar en cada uno el archivo de texto, las clases y los métodos solicitados.
2. Implementar cada ejercicio en Java (archivo .java), utilizando las clases y los métodos sugeridos en cada enunciado.
3. Crear previamente el archivo estudiantes.txt, y compilar y ejecutar los programas en orden, ya que el Ejercicio 3 elimina el archivo utilizado en el Ejercicio 2.
4. Comprobar que la salida de cada programa sea la esperada y realizar la prueba de escritorio de cada ejercicio, anotando el estado de las variables, del archivo y la salida en cada paso.
5. Registrar los resultados, redactar las conclusiones y recomendaciones y completar el informe de la guía práctica.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:
- Computadora personal con sistema operativo de escritorio.
- Java Development Kit (JDK) para compilar y ejecutar los programas .java.
- Editor de código o IDE (NetBeans, IntelliJ IDEA, Visual Studio Code o similar).
- Documento APE_5_primera_parte.docx con los enunciados de los ejercicios.

- Archivos de texto de prueba curso.txt y estudiantes.txt.
- Procesador de texto (Microsoft Word) para elaborar el informe.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☐Simuladores y laboratorios virtuales
☒Aplicaciones educativas
☐Recursos audiovisuales
☒Gamificación
☒Inteligencia Artificial
Otros (Especifique): _____

2.6 Actividades por desarrollar

Ejercicio 1 – Crear un fichero: crear el archivo curso.txt, comprobar si se creó correctamente o si ya existe, mostrar su nombre y controlar los errores con try-catch.
Ejercicio 2 – Escribir información en un fichero: ingresar un mensaje por teclado y agregarlo al archivo existente estudiantes.txt, conservando la información anterior y confirmando que se guardó.
Ejercicio 3 – Eliminar un fichero: eliminar el archivo estudiantes.txt y mostrar un mensaje que indique si se eliminó o si no se pudo eliminar.

2.7 Resultados obtenidos

. PRUEBA DE ESCRITORIO

Ejercicio 1: Crear un fichero (Creación de archivos)

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	File archivo = new File("curso.txt")	archivo → "curso.txt"; el archivo aún no existe en el disco	—
2	archivo.createNewFile() (primera ejecución)	curso.txt no existía: se crea vacío y el método devuelve true	—
3	if (true): System.out.println(...)	Entra en la rama if	Archivo creado correctamente.
4	archivo.getName()	Devuelve "curso.txt"	Nombre del archivo: curso.txt
5	archivo.createNewFile() (segunda ejecución)	curso.txt ya existe: no se modifica y el método devuelve false	—
6	else: System.out.println(...)	Entra en la rama else	El archivo ya existe.
7	archivo.getName()	Devuelve "curso.txt"	Nombre del archivo: curso.txt

Ejercicio 2: Escribir información en un fichero (Escritura en archivos)

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	Scanner teclado = new Scanner(System.in)	estudiantes.txt contiene: Primer Nivel - Paralelo B	—
2	System.out.print("Ingrese un mensaje: ")	Espera el dato del usuario	Ingrese un mensaje:
3	mensaje = teclado.nextLine()	mensaje = "Hola, soy estudiante de Software"	—
4	new FileWriter("estudiantes.txt", true)	Abre el archivo en modo agregar; se conserva el contenido anterior	—
5	escritor.write(mensaje + System.lineSeparator())	El mensaje se escribe al final, con un salto de línea	—
6	escritor.close()	Guarda y cierra; estudiantes.txt tiene 2 líneas	—
7	System.out.println(...)	Confirma la operación	Mensaje guardado correctamente.
8	teclado.close()	Se cierra el Scanner	—

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
9	Segunda ejecución con mensaje = "Hoy aprendimos a manejar ficheros"	estudiantes.txt tiene 3 líneas; las anteriores se conservan	Mensaje guardado correctamente.

Ejercicio 3: Eliminar un fichero (Eliminación de archivos)

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	File archivo = new File("estudiantes.txt")	archivo → "estudiantes.txt"; el archivo existe en el disco	—
2	archivo.delete() (primera ejecución)	estudiantes.txt existía: se elimina y el método devuelve true	—
3	if (true): System.out.println(...)	Entra en la rama if	Archivo eliminado correctamente.
4	archivo.delete() (segunda ejecución)	estudiantes.txt ya no existe: el método devuelve false	—
5	else: System.out.println(...)	Entra en la rama else	No se pudo eliminar el archivo. Verifique que exista.

2.8 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☐ Trabajo en equipo
☐ Comunicación asertiva
☐ La empatía
☐ Pensamiento crítico
☒ Flexibilidad
☒ La resolución de conflictos
☒ Adaptabilidad
☒ Responsabilidad

2.9 Conclusiones

- Se implementaron los tres ejercicios en Java y cada programa produjo la salida esperada, lo que confirma que las clases File y FileWriter permiten crear, escribir y eliminar ficheros de texto de forma sencilla.
- El modo agregar (parámetro true en FileWriter) permitió añadir información al final de estudiantes.txt sin perder el contenido anterior, y close() permitió guardar los datos y liberar el archivo.
- Los valores que devuelven createNewFile() y delete(), junto con el uso de try-catch, permitieron informar al usuario tanto en los casos válidos como en los de fallo, como se comprobó con las pruebas de escritorio.

2.10 Recomendaciones

- Crear el archivo estudiantes.txt antes de ejecutar el Ejercicio 2 y dejar el Ejercicio 3 para el final, porque elimina ese mismo archivo y la operación no se puede deshacer desde el programa.
- Cerrar siempre los archivos con close() después de escribir, para asegurar que la información se guarde y que el archivo quede disponible para otras operaciones, como su eliminación.
- Al escribir en un archivo, indicar expresamente el modo agregar con el parámetro true de FileWriter, ya que sin él se reemplaza el contenido anterior.

2.11 Referencias bibliográficas

Insertar las referencias bibliográficas empleadas aplicando la norma IEEE.

2.12 Anexos

Link del GITHUB: https://github.com/dereckflux/T2.4Fase1APE5.-Creaci-n-Lectura-y-Escritura-de-Ficheros

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

INFORME GENERAL DE EJERCICIOS

T2.4Fase1APE5. Creación, Lectura y Escritura de Ficheros

Dato	Detalle
Asignatura	Algoritmos y Lógica de Programación
Nivel y paralelo	Primero – B
Estudiante	Paredes Acosta Dereck Shair
Docente	Ing. José Ruben Caizabuano, M.Sc.
Lenguajes	Java y C++

1. Objetivo

Consolidar los tres ejercicios de manejo de ficheros de texto (crear, escribir y eliminar), implementados en los lenguajes Java y C++, y verificar su funcionamiento mediante pruebas de escritorio.

2. Resumen de los ejercicios

Ejercicio	Archivo de texto	Clases y métodos principales
1. Crear un fichero	curso.txt	Java: File, createNewFile(), getName(). C++: ifstream, ofstream
2. Escribir información	estudiantes.txt	Java: Scanner, FileWriter, write(), close(). C++: getline, ofstream con ios::app
3. Eliminar un fichero	estudiantes.txt	Java: File, delete(). C++: remove()

3. Ejercicio 1: Crear un fichero

3.1 Enunciado

Desarrollar un programa en Java que permita crear un archivo de texto llamado curso.txt.

Además del programa en Java, se desarrolló su versión equivalente en C++.

3.2 Análisis y diseño de clases

Clase o biblioteca	Objetos o variables	Métodos utilizados
File (Java)	archivo: File	createNewFile(): boolean; getName(): String
IOException (Java)	e: IOException	Excepción capturada en el bloque catch
ifstream, ofstream (C++)	comprobar: ifstream; archivo: ofstream	is_open(): bool; close()
exception (C++)	e: const exception&	Excepción capturada en el bloque catch

3.3 Código en Java

Archivo: Ejercicio1.java

```
// Ejercicio 1: Crear un fichero
import java.io.File;
import java.io.IOException;

public class Ejercicio1 {
    public static void main(String[] args) {
        try {
            // Objeto File que representa el archivo curso.txt
            File archivo = new File("curso.txt");

            // createNewFile() devuelve true si crea el archivo y false si ya existe
            if (archivo.createNewFile()) {
                System.out.println("Archivo creado correctamente.");
            } else {
                System.out.println("El archivo ya existe.");
            }

            System.out.println("Nombre del archivo: " + archivo.getName());
        } catch (IOException e) {
            System.out.println("Se produjo un error al crear el archivo.");
        }
    }
}
```

3.4 Código en C++

Archivo: Ejercicio1.cpp

```
// Ejercicio 1: Crear un fichero
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

int main() {
    string nombre = "curso.txt";

    try {
        // Si el archivo se puede abrir para lectura, significa que ya existe
        ifstream comprobar(nombre);

        if (comprobar.is_open()) {
            cout << "El archivo ya existe." << endl;
            comprobar.close();
        } else {
            // ofstream crea el archivo cuando no existe
            ofstream archivo(nombre);
            if (!archivo.is_open()) {
```

```
                throw runtime_error("No se pudo crear el archivo");
            }
            archivo.close();
            cout << "Archivo creado correctamente." << endl;
        }

        cout << "Nombre del archivo: " << nombre << endl;
    } catch (const exception& e) {
        cout << "Se produjo un error al crear el archivo." << endl;
    }

    return 0;
}
```

3.5 Prueba de escritorio

La prueba sigue la versión en Java; en C++ se realizan los mismos pasos con ifstream para comprobar el archivo y ofstream para crearlo.

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	File archivo = new File("curso.txt")	archivo → "curso.txt"; el archivo aún no existe en el disco	—
2	archivo.createNewFile() (primera ejecución)	curso.txt no existía: se crea vacío y el método devuelve true	—
3	if (true): System.out.println(...)	Entra en la rama if	Archivo creado correctamente.
4	archivo.getName()	Devuelve "curso.txt"	Nombre del archivo: curso.txt
5	archivo.createNewFile() (segunda ejecución)	curso.txt ya existe: no se modifica y el método devuelve false	—
6	else: System.out.println(...)	Entra en la rama else	El archivo ya existe.
7	archivo.getName()	Devuelve "curso.txt"	Nombre del archivo: curso.txt

3.6 Resultado de la ejecución

El programa se ejecutó dos veces en cada lenguaje. La salida obtenida fue idéntica en Java y en C++:

```
Primera ejecución:
Archivo creado correctamente.
Nombre del archivo: curso.txt

Segunda ejecución:
El archivo ya existe.
Nombre del archivo: curso.txt
```

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

4. Ejercicio 2: Escribir información en un fichero

4.1 Enunciado

Desarrollar un programa en Java que permita ingresar un mensaje por teclado y almacenarlo en un archivo de texto existente llamado estudiantes.txt.

Nota: El archivo estudiantes.txt debe existir previamente. Además del programa en Java, se desarrolló su versión equivalente en C++.

4.2 Análisis y diseño de clases

Clase o biblioteca	Objetos o variables	Métodos utilizados
Scanner (Java)	teclado: Scanner; mensaje: String	nextLine(): String; close()
FileWriter (Java)	escritor: FileWriter	FileWriter(nombre, true); write(texto); close()
IOException (Java)	e: IOException	Excepción capturada en el bloque catch
ofstream (C++)	escritor: ofstream; mensaje: string	ofstream(nombre, ios::app); is_open(): bool; close()
exception (C++)	e: const exception&	Excepción capturada en el bloque catch

4.3 Código en Java

Archivo: Ejercicio2.java

```
// Ejercicio 2: Escribir información en un fichero
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio2 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Ingrese un mensaje: ");
        String mensaje = teclado.nextLine();

        try {
            // El parámetro true conserva la información anterior
            FileWriter escritor = new FileWriter("estudiantes.txt", true);
            escritor.write(mensaje + System.lineSeparator());
            escritor.close();

            System.out.println("Mensaje guardado correctamente.");
        } catch (IOException e) {
            System.out.println("Se produjo un error al escribir en el archivo.");
        }

        teclado.close();
    }
}
```

4.4 Código en C++

Archivo: Ejercicio2.cpp

```
// Ejercicio 2: Escribir información en un fichero
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

int main() {
    string mensaje;

    cout << "Ingrese un mensaje: ";
    getline(cin, mensaje);

    try {
```

```
        // ios::app conserva la información anterior
        ofstream escritor("estudiantes.txt", ios::app);
        if (!escritor.is_open()) {
            throw runtime_error("No se pudo abrir el archivo");
        }

        escritor << mensaje << endl;
        escritor.close();

        cout << "Mensaje guardado correctamente." << endl;
    } catch (const exception& e) {
        cout << "Se produjo un error al escribir en el archivo." << endl;
    }

    return 0;
}
```

4.5 Prueba de escritorio

La prueba sigue la versión en Java; en C++ se realizan los mismos pasos con getline y ofstream en modo ios::app.

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	Scanner teclado = new Scanner(System.in)	estudiantes.txt contiene: Primer Nivel - Paralelo B	—
2	System.out.print("Ingrese un mensaje: ")	Espera el dato del usuario	Ingrese un mensaje:
3	mensaje = teclado.nextLine()	mensaje = "Hola, soy estudiante de Software"	—
4	new FileWriter("estudiantes.txt", true)	Abre el archivo en modo agregar; se conserva el contenido anterior	—
5	escritor.write(mensaje + System.lineSeparator())	El mensaje se escribe al final, con un salto de línea	—
6	escritor.close()	Guarda y cierra; estudiantes.txt tiene 2 líneas	—
7	System.out.println(...)	Confirma la operación	Mensaje guardado correctamente.
8	teclado.close()	Se cierra el Scanner	—
9	Segunda ejecución con mensaje = "Hoy aprendimos a manejar ficheros"	estudiantes.txt tiene 3 líneas; las anteriores se conservan	Mensaje guardado correctamente.

4.6 Resultado de la ejecución

Antes de la prueba, estudiantes.txt contenía la línea Primer Nivel - Paralelo B. El programa se ejecutó dos veces en cada lenguaje. La salida obtenida fue idéntica en Java y en C++:

```
Primera ejecución:
Ingrese un mensaje: Hola, soy estudiante de Software
Mensaje guardado correctamente.

Segunda ejecución:
Ingrese un mensaje: Hoy aprendimos a manejar ficheros
Mensaje guardado correctamente.

Contenido final de estudiantes.txt:
Primer Nivel - Paralelo B
Hola, soy estudiante de Software
Hoy aprendimos a manejar ficheros
```

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

5. Ejercicio 3: Eliminar un fichero

5.1 Enunciado

Desarrollar un programa en Java que permita eliminar el archivo de texto estudiantes.txt.

Además del programa en Java, se desarrolló su versión equivalente en C++.

5.2 Análisis y diseño de clases

Clase o biblioteca	Objetos o variables	Métodos utilizados
File (Java)	archivo: File	delete(): boolean
Biblioteca cstdio (C++)	—	remove(nombre): int (devuelve 0 si elimina el archivo)

5.3 Código en Java

Archivo: Ejercicio3.java

```
// Ejercicio 3: Eliminar un fichero
import java.io.File;

public class Ejercicio3 {
    public static void main(String[] args) {
        // Objeto File que representa el archivo estudiantes.txt
        File archivo = new File("estudiantes.txt");

        // delete() devuelve true si elimina el archivo y false si no pudo hacerlo
        if (archivo.delete()) {
            System.out.println("Archivo eliminado correctamente.");
        } else {
            System.out.println("No se pudo eliminar el archivo. Verifique que exista.");
        }
    }
}
```

5.4 Código en C++

Archivo: Ejercicio3.cpp

```
// Ejercicio 3: Eliminar un fichero
#include <cstdio>
#include <iostream>
using namespace std;

int main() {
    // remove() devuelve 0 si elimina el archivo
    if (remove("estudiantes.txt") == 0) {
        cout << "Archivo eliminado correctamente." << endl;
    } else {
        cout << "No se pudo eliminar el archivo. Verifique que exista." << endl;
    }

    return 0;
}
```

5.5 Prueba de escritorio

La prueba sigue la versión en Java; en C++ se realizan los mismos pasos con la función remove(), que devuelve 0 si elimina el archivo.

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	File archivo = new File("estudiantes.txt")	archivo → "estudiantes.txt"; el archivo existe en el disco	—
2	archivo.delete() (primera ejecución)	estudiantes.txt existía: se elimina y el método devuelve true	—
3	if (true): System.out.println(...)	Entra en la rama if	Archivo eliminado correctamente.

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
4	archivo.delete() (segunda ejecución)	estudiantes.txt ya no existe: el método devuelve false	—
5	else: System.out.println(...)	Entra en la rama else	No se pudo eliminar el archivo. Verifique que exista.

5.6 Resultado de la ejecución

Se creó estudiantes.txt antes de la prueba y el programa se ejecutó dos veces en cada lenguaje. La salida obtenida fue idéntica en Java y en C++:

```
Primera ejecución:
Archivo eliminado correctamente.

Segunda ejecución:
No se pudo eliminar el archivo. Verifique que exista.
```